
Yuzie — Product & Engineering Specification

| A collaborative, Git-aware project board for the terminal

Product & Engineering Spec · v1.0
19 August 2026

Contents

1. Document Control

2. Executive Summary

- 2.1 What success looks like for v1

3. Problem & Motivation

- 3.1 The context-switch tax
- 3.2 Why existing terminal tools don't solve it
- 3.3 Why now

4. Users & Personas

- 4.1 Primary persona — “Rahul”, the terminal-resident engineer
- 4.2 Secondary persona — “Priya”, the tech lead
- 4.3 Tertiary persona — “Adarsh”, the occasional contributor
- 4.4 Fourth participant — the agent
- 4.5 Anti-persona

5. Product Principles & Scope

- 5.1 Principles
- 5.2 In scope for v1.0
- 5.3 Explicitly out of scope for v1.0
- 5.4 Version horizon

6. End-to-End User Journeys

- 6.1 Journey A — First run in an existing repo
- 6.2 Journey B — Daily loop for an engineer
- 6.3 Journey C — Lead checking the board's true state
- 6.4 Journey D — Two people, live
- 6.5 Journey E — Agent claims a card
- 6.6 Journey F — Jumping straight into the code

7. CLI Specification

- 7.1 Invocation model
- 7.2 Command surface
- 7.3 Output contract
- 7.4 Exit codes
- 7.5 Card ID resolution

8. TUI Specification

- 8.1 Design goals
- 8.2 Board view
- 8.3 Card detail view
- 8.4 Keybindings
- 8.5 Presence rendering
- 8.6 Empty, error, and degraded states

9. Git Integration — The Differentiator

- 9.1 What a card derives from Git
- 9.2 Branch naming
- 9.3 `yuzie claim` — precise semantics
- 9.4 `yuzie finish` — pre-flight
- 9.5 Git hooks
- 9.6 Commit → card resolution order
- 9.7 Code anchors

10. System Architecture

- 10.1 Topology
- 10.2 Monorepo layout
- 10.3 Technology decisions
- 10.4 Non-functional requirements

11. Data Model

- 11.1 Entities
- 11.2 Schema (Postgres dialect; SQLite-compatible subset)
- 11.3 Local cache schema (SQLite, `.yuzie/cache/yuzie.db`)
- 11.4 Ordering and conflict strategy

12. API Specification

- 12.1 REST endpoints
- 12.2 WebSocket protocol
- 12.3 Event catalogue

13. SDK, Configuration, and Extensibility

- 13.1 `@yuzie/sdk`
- 13.2 `.yuzie/config.json` (per repo, committed)
- 13.3 Credentials
- 13.4 MCP server for agents (`yuzie mcp`)
- 13.5 Shell integration

14. Security & Privacy

- 14.1 Threat model
- 14.2 Permissions matrix
- 14.3 Privacy commitments

15. Observability

16. Testing Strategy

17. Packaging & Distribution

18. Build Plan — 18 Claude Code Sessions

- 18.1 How to run these sessions
- Session 0 — Repository, toolchain, and CI
- Session 1 — `@yuzie/core`: types, schemas, events, reducer
- Session 2 — `@yuzie/store`: local SQLite cache and outbox
- Session 3 — `@yuzie/server`: REST API and persistence
- Session 4 — Realtime gateway: WebSocket, presence, replay
- Session 5 — `@yuzie/sdk`
- Session 6 — CLI skeleton, config, auth, `yuzie init`
- Session 7 — Core CLI commands
- Session 8 — TUI foundation: board view
- Session 9 — TUI card detail, modals, and editing
- Session 10 — Realtime in the TUI: live updates and presence
- Session 11 — `@yuzie/git`: repo introspection and the claim flow
- Session 12 — Code anchors, open-in-editor, open-in-browser
- Session 13 — Offline mode and the sync engine
- Session 14 — Watching, activity, notifications, search
- Session 15 — Agent integration: tokens and MCP server
- Session 16 — Hardening: performance, resilience, and diagnostics
- Session 17 — Docs, packaging, and release

19. Milestones

20. Risks & Mitigations

21. Post-v1 Roadmap

Appendix A — Full Command Reference (quick card)

Appendix B — Keybinding Cheatsheet

Appendix C — Error Codes

Appendix D — Glossary

Appendix E — Definition of Done (applies to every session)

1. Document Control

FIELD	VALUE
Product name	Yuzie
CLI binary	yuzie (alias yz)
Distribution	npm package, runnable via npx yuzie
Package scope	@yuzie/* on npm
Spec version	1.0
Date	19 August 2026
Status	Approved for build
Primary implementation agent	Claude Code
Primary language	TypeScript (Node.js 20+)
Intended readers	Implementing engineers, Claude Code sessions, future contributors

How to use this document. Sections 1–13 are the *product and architecture contract*: what gets built and why. Section 14 is the *build plan*, split into 18 discrete Claude Code sessions. Each session is self-contained, has explicit inputs, deliverables and acceptance criteria, and ends with a copy-pasteable prompt. Do not start a session until its dependencies are green. Appendices are reference material that both humans and the coding agent should treat as normative.

2. Executive Summary

Yuzie is a collaborative kanban board that lives in the terminal and understands your Git repository.

Developers spend their day in the terminal and the editor. Project tracking lives somewhere else — a browser tab that costs a context switch every time. Existing terminal task tools solve this by being single-player and Git-ignorant: a local TODO file with nicer rendering. They break the moment a second person joins.

Yuzie takes the opposite position. It is **multiplayer first** and **Git-aware first**:

- Every teammate connected to the same board sees changes instantly — card moves, assignments, comments, and who is currently working on what.
- Cards are bound to real repository state. A card knows its branch, its commit count, its changed files, its PR, and the exact file and line where the work lives.
- `yuzie claim 18` assigns the card, creates `task/18-fix-github-oauth`, checks it out, and broadcasts “@rahul is working on #18” to everyone else’s terminal.

It ships as a **CLI first, TUI second**. Every operation is available as a one-shot command that composes with shell pipelines and scripts (`yuzie list --json | jq`). Running `yuzie` with no arguments opens the full interactive board for people who want to browse, drag, and read.

Underneath both is a typed SDK (`@yuzie/sdk`) published alongside the CLI, so a VS Code extension, a web dashboard, a CI job, or an AI agent can all drive the same board.

Positioning statement

Yuzie is Trello for developers who never leave the terminal — except it knows what branch you're on.

One-line pitch for the README

A real-time, Git-aware kanban board for your team, in your terminal. `npx yuzie`.

2.1 What success looks like for v1

DIMENSION	TARGET
Time to first board	< 60 seconds from <code>npx yuzie init</code> for a new user
Latency of a change appearing on a teammate's screen	< 250 ms p95 on a normal connection
Cold start of the TUI	< 400 ms to first paint
Team size supported per board	25 concurrent connections without degradation
Works offline	Yes — read + queued writes, reconciled on reconnect
Non-goals shipped	Zero (see §5.3)

3. Problem & Motivation

3.1 The context-switch tax

A developer's working loop is: read task → write code → commit → mark progress. Three quarters of that loop is in the terminal and the editor. The fourth step is in a browser, behind a login, inside a JavaScript app that takes two seconds to load a board. The cost is not the two seconds; it's the tab that stays open, the notifications it brings, and the twenty minutes that evaporate afterwards.

The common workarounds are all bad:

1. **Nobody updates the board.** Status is discovered in standup, badly.
2. **A local TODO.md.** Works for one person, invisible to the team, drifts from reality.
3. **The PR is the board.** Only captures work that already has code, and only in one direction.

3.2 Why existing terminal tools don't solve it

TOOL CLASS	WHY IT FALLS SHORT
Local TUI todo apps (<code>taskwarrior</code> , <code>kanban.nvim</code> , plaintext)	Single player. No shared state, no presence, no notion of “who is on this”.
Issue-tracker CLIs (<code>gh issue</code> , <code>jira-cli</code>)	Thin wrappers over a web product. Request/response, no live board, no presence, no local board model.
Chat-driven standups	Ephemeral, unstructured, not linked to code.
Full web trackers	Correct feature set, wrong surface for the user we're targeting.

The gap Yuzie fills is a tool that is **simultaneously** terminal-native, multiplayer, and repository-aware. None of the three alone is novel. The combination is the product.

3.3 Why now

- Terminal UIs are cheap to build well (Ink, modern truecolor terminals, wide Nerd Font adoption).
- `npk` makes zero-install distribution to developers trivial.
- AI coding agents have become a *new class of board participant* — an agent that can claim a card, open a branch, and report progress needs exactly this API surface. Yuzie is designed so an agent is a first-class assignee, not a bolt-on.

4. Users & Personas

4.1 Primary persona — “Rahul”, the terminal-resident engineer

- 3–8 years experience, works in tmux + Neovim/VS Code, lives on the keyboard.
- Part of a 3–8 person product team shipping continuously.
- Actively hostile to process tooling that costs him keystrokes.
- **Job to be done:** “Tell me what I should be working on, let me claim it in one command, and update everyone without me writing a status message.”

4.2 Secondary persona — “Priya”, the tech lead

- Runs a small team, needs visibility without running a meeting.
- Cares about: what’s in review, what’s blocked, what’s been claimed for two days with zero commits.
- **Job to be done:** “Let me see the true state of the sprint, derived from the repo rather than from what people remembered to update.”

4.3 Tertiary persona — “Adarsh”, the occasional contributor

- Designer, PM, or part-time contributor who doesn’t live in the terminal.
- Needs read access and the ability to file and comment.
- **Job to be done:** “Let me drop a task into the team’s board without learning their tooling.” → served by the SDK-backed web read view (v2) and by `yuzie` running in its simplest interactive mode.

4.4 Fourth participant — the agent

- An AI coding agent (Claude Code, or a CI bot) authenticated with a scoped token.
- Can be assigned cards, can comment, can move cards, can attach commits.
- Appears in presence as `@claude ● working (agent)`.
- **Job to be done:** “Let a human hand me a card, and let the humans watch my progress in the same place they watch each other’s.”

4.5 Anti-persona

Enterprise program managers who need epics, portfolios, story points, burndown charts, custom workflows, and permission matrices. Yuzie will never serve them well and should not try.

5. Product Principles & Scope

5.1 Principles

1. **CLI is the product; the TUI is a client.** Every capability must be reachable non-interactively. If a feature can only be done by arrow keys, it's a design bug.
2. **Scriptable by default.** Every read command supports `--json`. Every write command returns a meaningful exit code. `yuzie` composes with `jq`, `fzf`, git hooks, and CI.
3. **The repository is the source of truth for code state.** Yuzie never asks a human to type what Git already knows: branch, commits, files changed, PR status.
4. **Optimistic and local-first.** Actions apply instantly to the local view and reconcile with the server. Latency is never a reason to hesitate before pressing a key.
5. **Presence over notifications.** Ambient awareness ("Rahul is working on #18") beats a notification feed.
6. **Boring, legible protocol.** JSON over HTTP and WebSocket, an append-only event log, monotonic sequence numbers. No CRDT unless the data model demands it (it doesn't — see §11.4).
7. **Zero-config first run.** `npx yuzie` in a Git repo should produce something useful before the user reads any docs.
8. **Agents are users.** Anything a human can do over the API, a scoped token can do.

5.2 In scope for v1.0

- Workspaces, boards, columns, cards, ordering.
- Users, invites, roles (owner / member / viewer).
- Assignment, comments, checklists, labels, due dates.
- Real-time sync + presence + "currently viewing / working".
- Full CLI command surface (§7).
- Interactive TUI: board view, card detail, quick actions, search (§8).
- Git integration: repo detection, branch linking, `claim / start / finish`, commit attribution, changed-file counts, PR link (§9).
- Code anchors (`file:line`) and open-in-editor / open-in-browser.
- Offline read + queued writes with reconciliation.
- Typed SDK `@yuzie/sdk`.
- Self-hostable server (Docker Compose) and a hosted default.
- Agent tokens + a `yuzie mcp` server so coding agents can drive the board.

5.3 Explicitly out of scope for v1.0

NOT BUILDING	WHY
Web UI	Terminal is the wedge. SDK makes it cheap later.
Swimlanes, epics, sprints, story points	Trello-clone trap. Adds model complexity, serves the anti-persona.
Custom workflow engines / automation builder	v2 at the earliest; <code>yuzie hooks</code> covers 80% of it.
File attachments (binary upload)	Links only in v1. Storage cost and scope.
Two-way GitHub Issues sync	One-way <i>link</i> only in v1. Bidirectional sync is a product on its own.
Time tracking	Derivable from events later.
Mobile / notifications to phone	No.
End-to-end encryption	Server needs to read card state for search and derived views. Documented honestly (§14).

5.4 Version horizon

- **v1.0** — everything in §5.2.
- **v1.1** — GitHub Issues one-way import, saved filters, board templates, `yuzie stats`.
- **v1.5** — read-only web view generated from SDK, VS Code extension.

- **v2.0** — bidirectional issue sync, automation rules, multi-repo boards.

6. End-to-End User Journeys

These journeys are the acceptance narrative for the whole product. Every one of them must work end-to-end before v1.0 ships.

6.1 Journey A — First run in an existing repo

```
$ cd ~/code/payments-api
$ npx yuzie init

yuzie · collaborative git-aware kanban

✓ Git repository detected: payments-api (github.com/acme/payments-api)
✓ Default branch: main

? Sign in with GitHub? (Y/n) y
→ Opening https://yuzie.dev/device and waiting...
  Code: WXYZ-4821
✓ Signed in as @rahul

? Board name: (payments-api)
? Columns: (Todo, Doing, Review, Done)

✓ Board "payments-api" created
✓ Wrote .yuzie/config.json
✓ Added .yuzie/cache/ to .gitignore
✓ Installed git hooks (post-commit, post-checkout)

Invite your team:
  yuzie invite adarsh@acme.dev
  yuzie share                # prints a join link

Next: yuzie add "Fix GitHub OAuth"
```

6.2 Journey B — Daily loop for an engineer

```
$ yuzie                                # opens the TUI, sees the board
                                       # arrow to #18, presses `c` to claim

$ yuzie claim 18
✓ #18 assigned to @rahul
✓ Moved to Doing
✓ Created branch task/18-fix-github-oauth
✓ Checked out task/18-fix-github-oauth
✓ Broadcast to 4 teammates

# ... writes code, commits normally ...
$ git commit -m "fix: handle oauth callback state mismatch"
[yuzie] linked commit a3f9c21 → #18

$ yuzie comment 18 "Callback was dropping the state param. Fixed."
$ yuzie move 18 review
✓ #18 → Review · @priya notified (watching)

$ yuzie finish 18
△ Branch task/18-fix-github-oauth has 2 uncommitted files
? Continue anyway? (y/N)
```

6.3 Journey C — Lead checking the board's true state

```
$ yuzie list --status doing --json | jq '[] | {id, title, assignee, staleDays}'
$ yuzie list --stale 2d

#21 Fix websocket reconnect @rahul Doing claimed 3d ago · 0 commits
#24 Rate limiter @adarsh Doing claimed 2d ago · 1 commit · no push

$ yuzie card 21
```

6.4 Journey D — Two people, live

Rahul's terminal, TUI open. Priya, elsewhere, runs `yuzie move 15 done`.

Within ~200 ms Rahul's board re-renders: card #15 slides to Done, and a transient toast appears at the bottom:

```
✓ @priya moved #15 Login flow → Done just now
● 4 online · 2 working · synced
```

6.5 Journey E — Agent claims a card

```
$ yuzie assign 27 @claude
✓ #27 assigned to @claude (agent)

# Agent, running elsewhere with an agent token:
# yuzie claim 27 --agent
# yuzie comment 27 "Starting. Plan: 3 steps..."
# yuzie check 27 1 --done
# yuzie move 27 review

# Rahul sees, live:
● @claude ● working on #27 · 4 commits · 12 files
```

6.6 Journey F — Jumping straight into the code

```
$ yuzie card 18
...
Code      src/auth/oauth.ts:42
Branch    task/18-fix-github-oauth

$ yuzie open 18          # opens $EDITOR at src/auth/oauth.ts:42
$ yuzie open 18 --github  # opens the PR / branch compare view in browser
```

7. CLI Specification

7.1 Invocation model

```
yuzie                                # no args → launch TUI on the current board
yuzie <command> [args] [flags]
yz <command> [args] [flags] # `yz` is installed as a short alias
```

Global flags available on every command:

FLAG	EFFECT
--json	Machine-readable output. Suppresses all colour/spinners.
--board <slug>	Target a board other than the one in <code>.yuzie/config.json</code> .
--no-color	Disable ANSI colour (also honours <code>NO_COLOR</code>).
--quiet, -q	Only errors.
--verbose, -v	Debug logging to stderr.
--offline	Do not contact the server; operate on local cache, queue writes.
--yes, -y	Assume yes for all confirmations (for scripts and agents).
--config <path>	Override config file location.
--version, --help	Standard.

7.2 Command surface

Setup & auth

COMMAND	DESCRIPTION
yuzie init	Detect repo, authenticate, create or link a board, write config, install hooks.
yuzie login	Device-code auth flow; stores token in OS keychain (fallback: <code>~/.yuzie/credentials</code>).
yuzie logout	Revoke local token.
yuzie whoami	Print current user, workspace, board, connection state.
yuzie link <board-slug>	Attach the current repo to an existing board.
yuzie unlink	Detach repo from board (keeps board on server).

Boards

COMMAND	DESCRIPTION
yuzie boards	List boards you're a member of.
yuzie boards create <name>	Create a board.
yuzie boards rename <slug> <name>	Rename.
yuzie boards archive <slug>	Archive (soft delete).
yuzie columns	List columns for the current board.
yuzie columns add <name> [--after <name>]	Add a column.
yuzie columns rm <name>	Remove a column (must be empty).

Cards — the core

COMMAND	DESCRIPTION
yuzie add <title> [flags]	Create a card. Flags: <code>--desc</code> , <code>--assign @u</code> , <code>--column <c></code> , <code>--label <l></code> , <code>--due <date></code> , <code>--anchor <file:line></code> , <code>--priority <p></code> .
yuzie list [flags]	List cards. Flags: <code>--status/--column</code> , <code>--assignee @u</code> , <code>--label</code> , <code>--mine</code> , <code>--watching</code> , <code>--stale <dur></code> , <code>--search <q></code> , <code>--limit</code> , <code>--sort</code> .
yuzie card <id>	Full card detail view (read-only render).
yuzie move <id> <column>	Move a card. Column matched case-insensitively by prefix.
yuzie assign <id> @user [@user...]	Assign one or more people/agents. <code>--clear</code> to unassign.
yuzie claim <id>	Assign to self + move to Doing + create/checkout branch (\$9.3).
yuzie start <id>	Like claim but does not create a branch (or reuses an existing one).
yuzie finish <id>	Pre-flight checks, then move to the “done-ward” column.
yuzie done <id>	Move directly to the final column.
yuzie comment <id> <text>	Add a comment. <code>-</code> reads from stdin, <code>--editor</code> opens <code>\$EDITOR</code> .
yuzie edit <id>	Open card in <code>\$EDITOR</code> as YAML front-matter + markdown; save to apply a diff.
yuzie rm <id>	Delete a card (confirmation required).
yuzie watch <id> / yuzie unwatch <id>	Subscribe to a card's activity.
yuzie check <id> <n> [--done\ --undone]	Toggle checklist item <i>n</i> .
yuzie check <id> add <text>	Append a checklist item.
yuzie label <id> <label...>	Add labels. <code>--rm</code> to remove.
yuzie due <id> <date>	Set/clear due date (natural language accepted: <code>friday</code> , <code>+3d</code>).
yuzie priority <id> <p0..p3>	Set priority.

Git & code

COMMAND	DESCRIPTION
yuzie branch <id>	Print (and optionally create) the branch for a card.
yuzie anchor <id> <file:line>	Attach a code location to a card.
yuzie open <id>	Open the anchor in <code>\$EDITOR</code> . <code>--github</code> , <code>--browser</code> , <code>--pr</code> variants.
yuzie commits <id>	List commits linked to a card.
yuzie sync	Force a full reconcile (push queued writes, pull latest, re-scan Git).
yuzie hooks install\ uninstall	Manage the Git hooks.

Team & awareness

COMMAND	DESCRIPTION
yuzie invite <email\ @github>	Invite to the current board. <code>--role viewer\ member\ owner</code> .
yuzie members	List members with roles and last-seen.
yuzie who	Show presence: who's online, viewing, working on what.
yuzie activity [--since <dur>] [--card <id>]	Activity feed.
yuzie feed	Stream live events to stdout (blocking; great for a tmux pane).

Automation & integration

COMMAND	DESCRIPTION
yuzie token create --name <n> --role <r>	Mint a scoped token (for CI/agents).
yuzie mcp	Run an MCP server over stdio exposing board tools to an AI agent.
yuzie serve	Run the board server locally (self-host / dev).
yuzie export [--format json md csv]	Export the board.
yuzie import <file>	Import cards from JSON/CSV/markdown checklist.
yuzie completion <shell>	Emit shell completion script.

7.3 Output contract

Human mode. Colour, aligned columns, relative timestamps ("2m ago"), truncation to terminal width, symbols: ● working, ○ idle, ✓ done, ⚠ warning, → transition.

Example `yuzie list`:

#	TITLE	ASSIGNEE	COLUMN	BRANCH	ACT
18	Fix GitHub OAuth	@rahul ●	Doing	task/18-fix-github...	2m
21	Fix websocket reconnect	@adarsh	Doing	task/21-websocket	3h
13	Write tests for auth	@priya	Todo	—	1d
15	Login flow	@priya ✓	Done	task/15-login-flow	2d
4 cards · 3 online · synced					

JSON mode. Stable, versioned envelope. Never breaks without a major version bump.

```
{
  "apiVersion": "yuzie/v1",
  "kind": "CardList",
  "data": [
    {
      "id": 18,
      "title": "Fix GitHub OAuth",
      "column": "doing",
      "assignees": ["rahul"],
      "labels": ["bug", "auth"],
      "git": {
        "branch": "task/18-fix-github-oauth",
        "commits": 3,
        "filesChanged": 7,
        "lastActivityAt": "2026-08-19T09:14:22Z"
      },
      "anchor": { "path": "src/auth/oauth.ts", "line": 42 },
      "updatedAt": "2026-08-19T09:14:22Z"
    },
  ],
  "meta": { "count": 1, "boardSlug": "payments-api", "synced": true }
}
```

7.4 Exit codes

CODE	MEANING
0	Success
1	Generic runtime error
2	Usage error (bad flags/args)
3	Not authenticated
4	Not found (card/board/column)
5	Permission denied
6	Conflict (server rejected an optimistic write)
7	Offline and the operation requires the network
8	Git precondition failed (dirty tree, detached HEAD, etc.)
130	Interrupted (SIGINT)

7.5 Card ID resolution

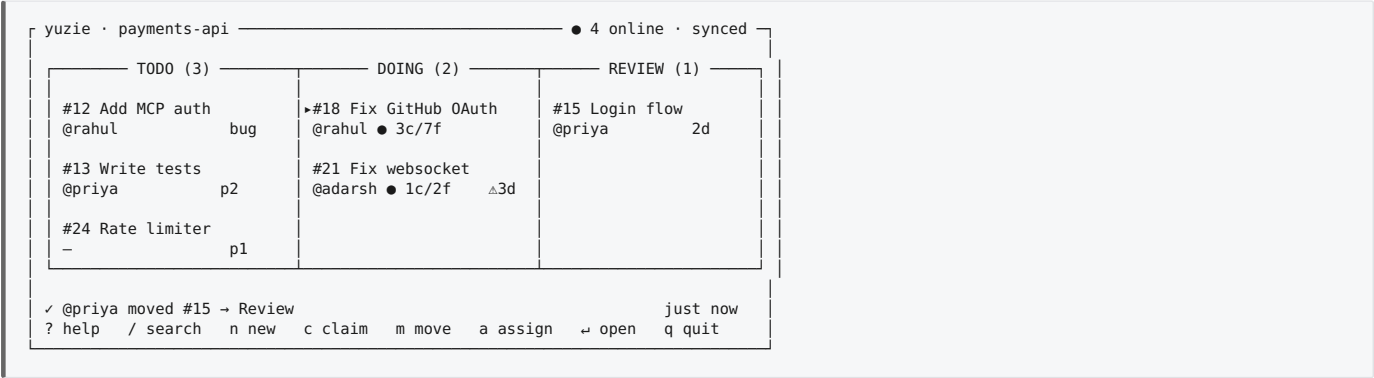
Card IDs are **per-board monotonic integers** shown as `#18`. Every command accepts `18`, `#18`, or a unique title prefix in interactive contexts. `yuzie move oauth doing` resolves fuzzily and asks for disambiguation if ambiguous (or fails with exit 4 under `--yes / --json`).

8. TUI Specification

8.1 Design goals

- First paint under 400 ms, including cached data.
- Fully keyboard-driven; mouse optional (click to select, scroll to pan).
- Never blocks on the network — render cache immediately, patch as data arrives.
- Degrade gracefully: 80×24 terminals, no truecolor, no Nerd Fonts.

8.2 Board view

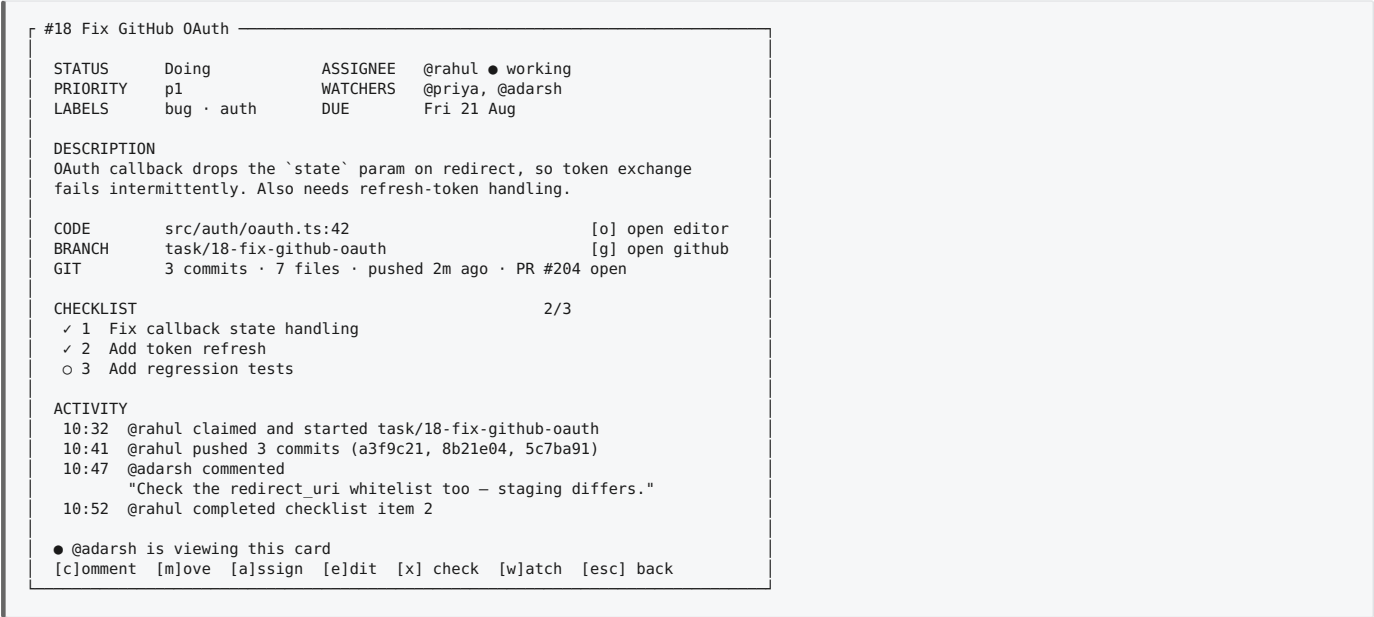


Layout rules:

- Columns are horizontally scrollable when they don't fit; a `< >` indicator shows overflow.
- Card cell shows: id, title (truncated), assignee(s), presence dot, git badge `3c/7f` (commits/files), staleness warning, priority/label chips.
- The selected card is marked with `▶` and inverse-video, not colour alone (accessibility).
- The footer alternates between the hint bar and the most recent live event toast (3 s), then reverts.

8.3 Card detail view

Opened with `Enter` or `yuzie card <id>`.



8.4 Keybindings

KEY	BOARD VIEW	CARD VIEW
<code>+/-</code> <code>h/l</code>	Move between columns	—
<code>↑/↓</code> <code>j/k</code>	Move between cards	Scroll
<code>Enter</code>	Open card	—
<code>Esc</code>	—	Back to board
<code>n</code>	New card (inline prompt)	—
<code>m</code>	Move card (column picker)	Same
<code>a</code>	Assign (member picker)	Same
<code>c</code>	Claim card	Same
<code>C</code>	Comment (opens input)	Same
<code>e</code>	Edit in <code>\$EDITOR</code>	Same
<code>x</code>	—	Toggle checklist item
<code>o</code>	Open code anchor in editor	Same
<code>g</code>	Open branch/PR in browser	Same
<code>w</code>	Watch/unwatch	Same
<code>d</code>	Mark done	Same
<code>D</code>	Delete (confirm)	Same
<code>/</code>	Search / filter	—
<code>f</code>	Filter menu (mine, label, assignee)	—
<code>r</code>	Force refresh/resync	Same
<code>?</code>	Help overlay	Same
<code>q</code> / <code>Ctrl-C</code>	Quit	Back

Vim-style `gg` / `G` jump to first/last card in a column. `1` – `9` jump to the *n*th column.

8.5 Presence rendering

STATE	RENDERING	SOURCE
Online, idle	<code>@user</code> (dim)	WS connection open
Viewing a card	<code>● @user is viewing</code> in card footer	<code>presence.view</code> event
Working (claimed + on branch)	<code>● @user</code> next to assignee, green	card status + branch checkout
Agent	<code>● @claude</code> (agent) in magenta	actor type
Recently pushed	<code>↑3</code> badge for 10 minutes	commit events

8.6 Empty, error, and degraded states

- **Empty board:** centred hint with the exact command to create the first card.
- **Disconnected:** header turns `△ offline · 2 queued`, the board stays interactive, writes queue.
- **Conflict:** the affected card flashes once and shows `⦿ updated by @priya`; local optimistic state is replaced by server state.
- **Narrow terminal (< 100 cols):** collapse to a single-column list view grouped by column, same keybindings.

9. Git Integration — The Differentiator

This is the feature that makes Yuzie a developer tool rather than a Trello skin. It must feel like the board *already knew*.

9.1 What a card derives from Git

FIELD	DERIVATION
branch	Explicit link stored on the card, created by <code>claim / start</code> or attached by <code>yuzie branch <id> --link <name></code>
commits	<code>git log <base>.. <branch></code> plus any commit whose message or trailer references <code>#<id></code>
filesChanged	<code>git diff --name-only <merge-base> <branch></code>
additions / deletions	<code>git diff --shortstat</code>
lastActivityAt	Latest committer date among linked commits
pushed	Whether the branch has an upstream and is not ahead
pr	Discovered via <code>gh</code> CLI if present, else via the GitHub API with the user's token, else null
dirty	Working tree state at the time of a <code>finish</code> pre-flight check

Git scanning is **local, lazy, and cached**. The CLI computes these on the client (it has the repo; the server doesn't) and pushes a `card.git.updated` event with the derived summary. The server stores the summary only; it never needs repo access. This keeps self-hosting trivial and avoids asking for repo scopes.

9.2 Branch naming

Default template, configurable in `.yuzie/config.json`:

```
task/{id}-{slugified-title}
```

- Title slug: lowercase, non-alphanumerics → `-`, collapsed, truncated to 40 chars, trailing `-` stripped.
- `#18 "Fix GitHub OAuth"` → `task/18-fix-github-oauth`.
- Alternative templates supported: `{user}/{id}-{slug}`, `feat/{id}-{slug}`, or a literal pattern with `{id}`, `{slug}`, `{user}`, `{column}`.
- If the branch already exists locally, `claim` checks it out instead of creating it. If it exists remotely, it fetches and tracks it.

9.3 `yuzie claim` — precise semantics

```
yuzie claim <id> [--no-branch] [--from <base>] [--force]
```

1. Resolve card; fail with exit 4 if not found.
2. Verify permission (member or owner) — exit 5 otherwise.
3. **Pre-flight Git checks:**
 - In a Git repo? Else warn and continue without branch operations.
 - Working tree clean? If dirty, prompt: stash / continue on current branch / abort. Under `--yes`, abort with exit 8 unless `--force`.
 - Detached HEAD? Abort with exit 8.
4. Optimistically apply locally: assignee += self, column → first “in progress” column.
5. Create branch from `--from` (default: the board's configured base branch, default `main`) and check out.
6. Emit events: `card.assigned`, `card.moved`, `card.branch.linked`, `presence.working`.
7. Print a receipt (see Journey B).
8. If any *server* step fails, the Git branch is still created; the CLI queues the writes and reports `⚠ queued (offline)`. Git is never rolled back silently.

9.4 yuzie finish — pre-flight

```
yuzie finish <id> [--skip-checks] [--push]
```

Checks, in order, each reported as ✓ / ⚠ :

- 1. Working tree clean.
- 2. Branch has at least one commit beyond base.
- 3. Branch pushed to upstream (offer --push).
- 4. All checklist items complete.
- 5. Tests: if checks.test is configured in .yuzie/config.json, run it and surface pass/fail.
- 6. PR exists (informational).

Any ⚠ prompts for confirmation. --skip-checks bypasses. Then move the card to the configured finishColumn (default Review, or Done if no review column exists).

9.5 Git hooks

yuzie hooks install writes (or appends to, preserving existing content) three hooks in .git/hooks, each a thin shell shim calling yuzie __hook <name> :

HOOK	BEHAVIOUR
post-commit	Parse commit message for #<id> ; else infer card from current branch name. Attach commit to card (async, non-blocking, 300 ms timeout). Print [yuzie] linked commit <sha> → #<id> .
post-checkout	If the new branch maps to a card, update presence to “working on #id”; if leaving, clear.
pre-push	Refresh the card’s git summary (commit count, files) so teammates see accurate numbers. Never blocks the push; failures are silent with --verbose exceptions.

Hard rule: hooks must never fail a Git operation. All hook logic is wrapped, time-limited, and exits 0 unconditionally.

9.6 Commit → card resolution order

- 1. Explicit trailer: Board-Card: 18 .
- 2. #18 anywhere in the subject or body.
- 3. Branch name matching {id} in the configured template.
- 4. Card currently claimed by this user and in an in-progress column, if exactly one.
- 5. Otherwise: unattributed (still recorded in a local buffer so yuzie sync can attribute later).

9.7 Code anchors

A card may hold one primary anchor and any number of secondary references:

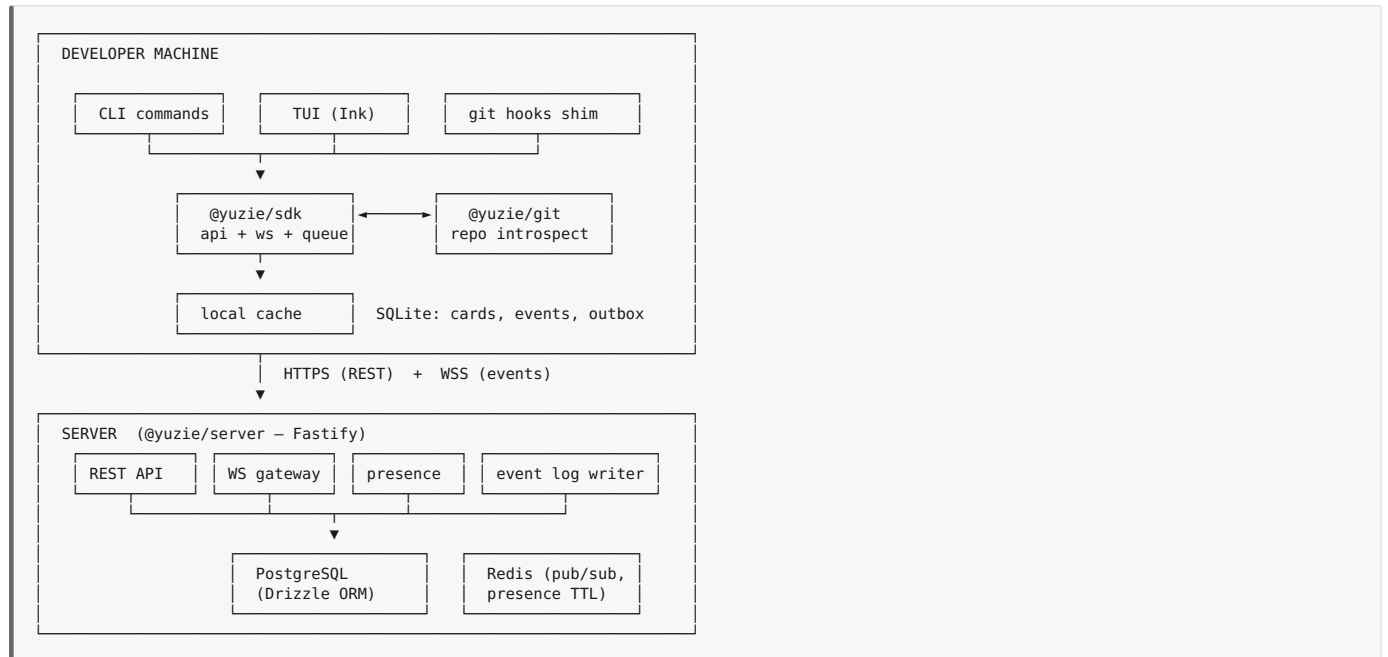
```
yuzie anchor 18 src/auth/oauth.ts:42
yuzie anchor 18 src/auth/oauth.ts:42-88      # range
yuzie open 18                                # $EDITOR at that line
```

Editor command resolution: \$YUZIE_EDITOR → \$VISUAL → \$EDITOR → detect (code, cursor, subl, nvim, vim) → fail with a helpful message. Line-jump syntax is per-editor and lives in a small table (code -g path:line, nvim +line path, etc.).

Anchors store path, line, endLine?, and the commitSha at which they were created, so a stale anchor can be reported as ⚠ anchor may be stale (file changed since) .

10. System Architecture

10.1 Topology



Redis is **optional** for single-node self-hosting (in-process pub/sub fallback) and required only for multi-node deployments. SQLite may replace Postgres for single-team self-hosting; the Drizzle schema targets both.

10.2 Monorepo layout



10.3 Technology decisions

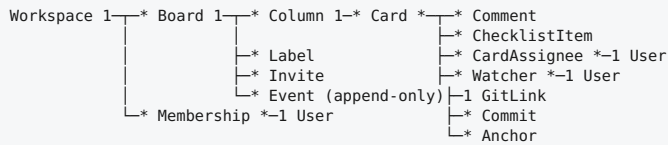
CONCERN	CHOICE	RATIONALE	REJECTED ALTERNATIVES
Language	TypeScript 5.x , Node 20+	One language across CLI, server, SDK; best terminal-UI ecosystem; <code>npx</code> distribution is native.	Go (better binaries, worse TUI/React ecosystem and no npm-native distribution); Rust (slowest to build for a solo/small team).
Python	Not used.	No component benefits. A Python client may ship post-v1 as a thin wrapper over the REST API.	—
CLI parsing	<code>commander</code>	Mature, small, good subcommand + help ergonomics.	<code>yargs</code> (heavier), <code>oclif</code> (too much framework).
TUI	<code>ink</code> (React for CLI) + <code>ink-testing-library</code>	Component model, diffing renderer, testable, hooks fit the realtime model.	<code>blessed</code> (unmaintained, imperative), raw ANSI (unmaintainable at this scope).
Prompts	<code>@inquirer/prompts</code>	Composes with commander for non-TUI flows.	—
Server	<code>fastify</code> + <code>@fastify/websocket</code>	Fast, schema-first (JSON Schema/zod), plugin model.	Express (slower, no schema), Nest (over-structured).
Validation	<code>zod</code>	Shared schemas between client, server, and SDK; single source of truth for the API contract.	io-ts, ajv-only.
ORM / DB	<code>drizzle-orm</code> + Postgres (prod) / SQLite (self-host, local cache)	Same query layer for both targets; SQL-first migrations.	Prisma (heavy binary, awkward for CLI bundling).
Local cache	<code>better-sqlite3</code>	Synchronous, fast, zero-daemon; ideal for a CLI.	JSON files (no query, no concurrency safety).
Realtime	Native WebSocket, JSON frames, monotonic <code>seq</code>	Simple, debuggable, replayable.	SSE (no upstream channel), gRPC (bad browser/CLI story), CRDT lib (unnecessary, \$11.4).
Auth	Device-code flow + PAT-style tokens in OS keychain	Works over SSH, no local browser callback needed.	OAuth redirect on localhost (breaks in remote/SSH/dev-container).
Keychain	<code>keytar</code> -alternative: <code>@napi-rs/keyring</code> , fallback to 0600 file	Avoids native build pain of <code>keytar</code> .	—
Bundling	<code>tsup</code> (esbuild)	Fast, produces a single-file CLI with a shebang.	Rollup (slower config), webpack (no).
Testing	<code>vitest</code> + <code>ink-testing-library</code> + <code>execa</code> for e2e	Same runner across packages, fast watch mode.	Jest (slower ESM story).
Release	<code>changesets</code> + GitHub Actions	Versioning across a monorepo of published packages.	Manual.
Lint/format	<code>biome</code>	One tool, fast, no plugin sprawl.	ESLint + Prettier.

10.4 Non-functional requirements

REQUIREMENT	TARGET	HOW VERIFIED
CLI cold start (one-shot command, cached)	< 150 ms	Benchmark in CI (hyperfine)
TUI first paint	< 400 ms	Timed integration test
Event propagation p95	< 250 ms	Load test with 25 simulated clients
Memory, TUI, 500-card board	< 120 MB RSS	Profiling test
Board size supported	2,000 cards, 25 concurrent clients	Load test
Offline behaviour	All reads served from cache; writes queued and reconciled	E2E test with network blackhole
Bundle size (CLI)	< 4 MB installed, < 2 s npx cold fetch	CI size budget check
Node support	20.x, 22.x, 24.x	Matrix CI
Platforms	macOS, Linux, WSL2; Windows Terminal best-effort	Matrix CI

11. Data Model

11.1 Entities



11.2 Schema (Postgres dialect; SQLite-compatible subset)

```
CREATE TABLE users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  handle      text UNIQUE NOT NULL,      -- 'rahul'
  email       text UNIQUE,
  display_name text,
  avatar_url  text,
  kind        text NOT NULL DEFAULT 'human', -- 'human' | 'agent'
  github_login text,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE workspaces (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug        text UNIQUE NOT NULL,
  name        text NOT NULL,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE boards (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id uuid NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  slug        text NOT NULL,
  name        text NOT NULL,
  repo_remote text,                    -- 'github.com/acme/payments-api'
  base_branch text DEFAULT 'main',
  branch_template text DEFAULT 'task/{id}-{slug}',
  next_card_no integer NOT NULL DEFAULT 1,
  archived_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (workspace_id, slug)
);

CREATE TABLE memberships (
  board_id  uuid NOT NULL REFERENCES boards(id) ON DELETE CASCADE,
  user_id   uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role      text NOT NULL,             -- 'owner' | 'member' | 'viewer'
  created_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (board_id, user_id)
);

CREATE TABLE columns (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  board_id    uuid NOT NULL REFERENCES boards(id) ON DELETE CASCADE,
  key         text NOT NULL,           -- 'todo' | 'doing' | 'review' | 'done'
  name        text NOT NULL,
  rank        text NOT NULL,          -- fractional index
  semantics   text,                   -- 'backlog' | 'in_progress' | 'review' | 'terminal'
  wip_limit   integer,
  UNIQUE (board_id, key)
);

CREATE TABLE cards (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  board_id    uuid NOT NULL REFERENCES boards(id) ON DELETE CASCADE,
  number      integer NOT NULL,       -- the '#18' users type
  column_id   uuid NOT NULL REFERENCES columns(id),
  rank        text NOT NULL,          -- ordering within column
  title       text NOT NULL,
  description text,
  priority    smallint,               -- 0..3
  due_at      timestampz,
  created_by  uuid REFERENCES users(id),
  archived_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  version     integer NOT NULL DEFAULT 1,
  UNIQUE (board_id, number)
);

CREATE INDEX ON cards (board_id, column_id, rank);

CREATE TABLE card_assignees (
  card_id  uuid REFERENCES cards(id) ON DELETE CASCADE,
  user_id  uuid REFERENCES users(id) ON DELETE CASCADE,
  PRIMARY KEY (card_id, user_id)
);
```

```

CREATE TABLE labels (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  board_id uuid REFERENCES boards(id) ON DELETE CASCADE,
  name    text NOT NULL,
  color   text,
  UNIQUE (board_id, name)
);

CREATE TABLE card_labels (
  card_id uuid REFERENCES cards(id) ON DELETE CASCADE,
  label_id uuid REFERENCES labels(id) ON DELETE CASCADE,
  PRIMARY KEY (card_id, label_id)
);

CREATE TABLE comments (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  card_id uuid NOT NULL REFERENCES cards(id) ON DELETE CASCADE,
  author_id uuid NOT NULL REFERENCES users(id),
  body    text NOT NULL,
  created_at timestampz NOT NULL DEFAULT now(),
  edited_at timestampz
);

CREATE TABLE checklist_items (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  card_id uuid NOT NULL REFERENCES cards(id) ON DELETE CASCADE,
  position integer NOT NULL,
  text    text NOT NULL,
  done_at timestampz,
  done_by uuid REFERENCES users(id)
);

CREATE TABLE watchers (
  card_id uuid REFERENCES cards(id) ON DELETE CASCADE,
  user_id uuid REFERENCES users(id) ON DELETE CASCADE,
  PRIMARY KEY (card_id, user_id)
);

CREATE TABLE git_links (
  card_id      uuid PRIMARY KEY REFERENCES cards(id) ON DELETE CASCADE,
  branch       text,
  base_branch  text,
  commit_count integer DEFAULT 0,
  files_changed integer DEFAULT 0,
  additions    integer DEFAULT 0,
  deletions    integer DEFAULT 0,
  pushed       boolean DEFAULT false,
  pr_url       text,
  pr_state     text,
  last_activity_at timestampz,
  updated_at   timestampz NOT NULL DEFAULT now()
);

CREATE TABLE commits (
  sha      text NOT NULL,
  card_id  uuid NOT NULL REFERENCES cards(id) ON DELETE CASCADE,
  author_id uuid REFERENCES users(id),
  message  text,
  committed_at timestampz,
  PRIMARY KEY (card_id, sha)
);

CREATE TABLE anchors (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  card_id uuid NOT NULL REFERENCES cards(id) ON DELETE CASCADE,
  path    text NOT NULL,
  line    integer,
  end_line integer,
  commit_sha text,
  primary_anchor boolean DEFAULT false
);

CREATE TABLE events (
  board_id uuid NOT NULL REFERENCES boards(id) ON DELETE CASCADE,
  seq      bigint NOT NULL,
  id        uuid NOT NULL,
  type      text NOT NULL,
  actor_id  uuid REFERENCES users(id),
  card_id   uuid,
  payload   jsonb NOT NULL,
  created_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (board_id, seq)
);

CREATE TABLE api_tokens (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  board_id uuid REFERENCES boards(id) ON DELETE CASCADE,
  name     text NOT NULL,
  token_hash text NOT NULL,
  role     text NOT NULL,
  last_used_at timestampz,
  expires_at timestampz,
  revoked_at timestampz
);

```

11.3 Local cache schema (SQLite, `.yuzie/cache/yuzie.db`)

Mirrors `cards`, `columns`, `comments`, `checklist_items`, `git_links` as denormalised tables, plus two client-only tables:

```
CREATE TABLE sync_state (  
  board_slug text PRIMARY KEY,  
  last_seq integer NOT NULL DEFAULT 0,  
  synced_at integer  
);  
  
CREATE TABLE outbox (  
  id integer PRIMARY KEY AUTOINCREMENT,      -- queued writes while offline  
  board_slug text NOT NULL,  
  op text NOT NULL,                          -- JSON: {method, path, body, idempotencyKey}  
  created_at integer NOT NULL,  
  attempts integer NOT NULL DEFAULT 0,  
  last_error text  
);
```

11.4 Ordering and conflict strategy

Ordering uses fractional indexing (lexicographic string ranks, LexoRank-style). Moving a card between two neighbours computes a rank strictly between them; no sibling rows are rewritten, so concurrent moves in different parts of a column never conflict. A rare rank collision triggers a lazy rebalance of that column.

Conflicts. The server is authoritative. The client applies optimistically and includes the card's `version` in mutating requests:

- Match → apply, `version++`, append event, broadcast.
- Mismatch → 409 with the current card state. The client replaces its optimistic state and shows `⌂ updated by @x`.
- Field-level merges are deliberately not attempted; cards are small and last-writer-wins on a *field group* (title/description vs column vs assignees are separate operations) is sufficient and predictable.

A CRDT is not used: the shared state is small, the operations are coarse, and a human-readable event log is worth far more here than automatic merge.

12. API Specification

Base URL `https://api.yuzie.dev/v1` (self-host: `http://localhost:8787/v1`). All requests carry `Authorization: Bearer <token>`. All bodies are JSON. All schemas are defined once in `@yuzie/core` with zod and reused by the server (validation) and SDK (types).

12.1 REST endpoints

METHOD	PATH	PURPOSE
POST	<code>/auth/device</code>	Start device-code flow → <code>{ deviceCode, userCode, verifyUrl, interval }</code>
POST	<code>/auth/device/token</code>	Poll for token
GET	<code>/me</code>	Current user + memberships
GET	<code>/boards</code>	List boards
POST	<code>/boards</code>	Create board
GET	<code>/boards/:slug</code>	Board with columns, labels, members
PATCH	<code>/boards/:slug</code>	Rename, set base branch/template
DELETE	<code>/boards/:slug</code>	Archive
GET	<code>/boards/:slug/cards</code>	List/filter cards
POST	<code>/boards/:slug/cards</code>	Create card
GET	<code>/boards/:slug/cards/:no</code>	Card detail (+ comments, checklist, git, anchors)
PATCH	<code>/boards/:slug/cards/:no</code>	Update fields (<code>If-Match: version</code>)
POST	<code>/boards/:slug/cards/:no/move</code>	<code>{ column, beforeId?, afterId? }</code>
POST	<code>/boards/:slug/cards/:no/assign</code>	<code>{ add: [], remove: [] }</code>
DELETE	<code>/boards/:slug/cards/:no</code>	Delete
POST	<code>/boards/:slug/cards/:no/comments</code>	Add comment
POST	<code>/boards/:slug/cards/:no/checklist</code>	Add item
PATCH	<code>/boards/:slug/cards/:no/checklist/:itemId</code>	Toggle/edit
PUT	<code>/boards/:slug/cards/:no/git</code>	Upsert derived git summary
POST	<code>/boards/:slug/cards/:no/commits</code>	Attach commits
PUT	<code>/boards/:slug/cards/:no/anchor</code>	Set anchor
POST	<code>/boards/:slug/cards/:no/watch</code>	Watch / unwatch
GET	<code>/boards/:slug/events?since=<seq>&limit=</code>	Replay events
GET	<code>/boards/:slug/presence</code>	Current presence snapshot
POST	<code>/boards/:slug/invites</code>	Invite a member
GET	<code>/boards/:slug/members</code>	List members
POST	<code>/tokens</code>	Create scoped token
DELETE	<code>/tokens/:id</code>	Revoke

Idempotency. Every mutating request accepts `Idempotency-Key`. The server stores the key → response for 24 h. This is what makes the offline outbox safe to retry.

Errors. Uniform envelope:

```
{
  "error": {
    "code": "card_not_found",
    "message": "Card #99 does not exist on board payments-api",
    "status": 404,
    "details": { "boardSlug": "payments-api", "number": 99 }
  }
}
```

Codes: `unauthenticated`, `forbidden`, `card_not_found`, `board_not_found`, `column_not_found`, `version_conflict`, `validation_failed`, `rate_limited`, `wip_limit_exceeded`, `internal`.

Rate limits. 600 req/min per token for reads, 120/min for writes, 429 with `Retry-After`.

12.2 WebSocket protocol

Connect: `wss://api.yuzie.dev/v1/boards/:slug/stream?since=<lastSeq>`, Authorization via `Sec-WebSocket-Protocol` `bearer` sub-protocol.

Frames — client → server

```
{ "t": "hello", "lastSeq": 4211, "client": "cli/1.0.0", "caps": ["presence"] }
{ "t": "presence", "state": "viewing", "cardNo": 18 }
{ "t": "presence", "state": "working", "cardNo": 18, "branch": "task/18-..." }
{ "t": "ping" }
```

Frames — server → client

```
{ "t": "welcome", "seq": 4211, "presence": [ ... ], "resumed": true }
{ "t": "snapshot", "seq": 4300, "board": { ... } } // sent when the gap is too large
{ "t": "event", "seq": 4212, "type": "card.moved",
  "actor": "priya", "cardNo": 15,
  "payload": { "from": "review", "to": "done", "rank": "a0m" },
  "ts": "2026-08-19T09:20:11Z" }
{ "t": "presence", "users": [ { "handle": "rahul", "state": "working", "cardNo": 18 } ] }
{ "t": "pong" }
```

Rules

- `seq` is strictly monotonic per board. A client that receives `seq > lastSeq + 1` requests a replay via `GET /events?since=`; if the gap exceeds 500 events, the server sends a `snapshot` instead.
- Reconnect uses exponential backoff with jitter: $0.5\text{ s} \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8\text{ s}$, capped at 30 s. The TUI shows `⚠ reconnecting...` after the first failure, never freezes.
- Heartbeat: client pings every 20 s; server closes after 45 s of silence. Presence entries expire 60 s after the last heartbeat.
- The client's own events echo back and are deduplicated by `idempotencyKey`, so optimistic state resolves cleanly.

12.3 Event catalogue

TYPE	PAYLOAD
card.created	full card
card.updated	changed fields + version
card.moved	{ from, to, rank }
card.assigned	{ added: [], removed: [] }
card.deleted	{ number }
comment.created	{ commentId, body, author }
checklist.updated	{ itemId, done }
card.branch.linked	{ branch, base }
card.git.updated	{ commits, filesChanged, additions, deletions, pushed, prUrl }
card.commits.attached	{ shas: [] }
card.anchor.set	{ path, line }
member.joined / member.left	{ handle, role }
board.updated	changed fields
presence.*	transient, not persisted to the event log

Everything except `presence.*` is persisted, which means `yuzie activity` and the card Activity panel are simply projections over the event table. There is no separate audit log to maintain.

13. SDK, Configuration, and Extensibility

13.1 @yuzie/sdk

```
import { Yuzie } from "@yuzie/sdk";

const board = await Yuzie.connect("payments-api", {
  token: process.env.YUZIE_TOKEN, // or reads the local credential store
  baseUrl: "https://api.yuzie.dev/v1", // optional
  offline: "queue", // "queue" | "fail"
});

// Reads
const cards = await board.cards.list({ column: "doing", assignee: "rahul" });
const card = await board.cards.get(18);

// Writes
const created = await board.cards.create({
  title: "Fix OAuth",
  assignee: "rahul",
  column: "todo",
  labels: ["bug"],
});
await board.cards.move(18, "review");
await board.cards.assign(18, ["rahul"]);
await board.cards.comment(18, "OAuth callback is broken");
await board.cards.check(18, 3, true);

// Git
await board.cards.linkBranch(18, "task/18-fix-github-oauth");
await board.cards.updateGit(18, { commits: 3, filesChanged: 7 });

// Realtime
const unsubscribe = board.on("card.moved", (e) => {
  console.log(`${e.actor} moved #${e.cardNo}: ${e.payload.from} → ${e.payload.to}`);
});
board.on("presence", (p) => render(p));

await board.close();
```

Design rules for the SDK:

- Fully typed from the zod schemas in `@yuzie/core`; no `any` in the public surface.
- Works in Node and (with `fetch` / `WebSocket` globals) in the browser — this is what makes the future web UI and VS Code extension cheap.
- No CLI-specific concerns leak in (no chalk, no `process.exit`, no prompts).
- Errors are typed classes: `BoardError`, `NotFoundError`, `ConflictError`, `OfflineError`, `PermissionError`.
- The CLI must consume *only* the SDK. If the CLI needs a raw HTTP call, that's a missing SDK method.

13.2 .yuzie/config.json (per repo, committed)

```
{
  "version": 1,
  "board": "payments-api",
  "workspace": "acme",
  "server": "https://api.yuzie.dev/v1",
  "git": {
    "baseBranch": "main",
    "branchTemplate": "task/{id}-{slug}",
    "autoLinkCommits": true,
    "hooks": ["post-commit", "post-checkout", "pre-push"]
  },
  "flow": {
    "startColumn": "doing",
    "finishColumn": "review",
    "doneColumn": "done"
  },
  "checks": {
    "test": "pnpm test",
    "requireCleanTree": true,
    "requirePushed": false
  },
  "ui": {
    "theme": "dark",
    "compact": false,
    "showGitBadges": true
  }
}
```

Secrets never go here. `.yuzie/cache/` is gitignored automatically by `yuzie init`.

13.3 Credentials

Precedence: `YUZIE_TOKEN` env var → OS keychain entry (`yuzie:<server>`) → `~/.yuzie/credentials` (mode 0600). `yuzie whoami --token` never prints the token; `yuzie token create` prints it exactly once.

13.4 MCP server for agents (`yuzie mcp`)

Exposes the board to an AI coding agent over stdio using the Model Context Protocol. Tools exposed:

TOOL	DESCRIPTION
<code>board_list_cards</code>	Filterable card list
<code>board_get_card</code>	Full card context including git state and comments
<code>board_create_card</code>	Create
<code>board_move_card</code>	Move between columns
<code>board_comment</code>	Post a comment (agents should narrate progress)
<code>board_claim_card</code>	Claim + branch (requires <code>--allow-git</code>)
<code>board_update_checklist</code>	Tick items

Guardrails: the MCP server refuses destructive operations (`delete` , `archive`) unless started with `--allow-destructive` ; all agent actions are attributed to the agent user and appear in presence and activity with an (`agent`) marker so humans always know what was machine-driven.

13.5 Shell integration

- `yuzie completion bash|zsh|fish` emits completion including dynamic card IDs and column names from cache.
- A suggested `starship` /prompt snippet shows the current card when on a `task/*` branch.
- `yuzie feed` is designed to be run in a dedicated tmux pane as an ambient team activity ticker.

14. Security & Privacy

14.1 Threat model

ASSET	THREAT	MITIGATION
Board content	Unauthorised read by non-member	Every query scoped by membership; no board is public in v1
Auth token on disk	Local exfiltration	OS keychain first; file fallback 0600; token is board-scoped and revocable
Agent token	Over-privileged automation	Tokens carry a role and an optional board scope; destructive ops gated
Git hooks	Supply-chain via injected shell	Hooks are generated shims that only call the installed <code>yuzie</code> binary; content is idempotent and printed before writing
Server	Injection / mass assignment	zod validation on every route; Drizzle parameterised queries; explicit field allow-lists on PATCH
Transport	MITM	TLS enforced; <code>http://</code> allowed only for <code>localhost</code>
Replay	Duplicated writes	Idempotency keys with 24 h retention
DoS	Event flood from a client	Per-token rate limits; WS message rate cap; max 2 connections per user per board

14.2 Permissions matrix

ACTION	OWNER	MEMBER	VIEWER	AGENT (MEMBER ROLE)
Read board/cards	✓	✓	✓	✓
Create/edit/move cards	✓	✓	✗	✓
Comment	✓	✓	✓	✓
Assign others	✓	✓	✗	✓
Delete card	✓	✓ (own)	✗	only with <code>--allow-destructive</code>
Manage columns/labels	✓	✗	✗	✗
Invite / change roles	✓	✗	✗	✗
Archive board	✓	✗	✗	✗

14.3 Privacy commitments

- No repository `code` is ever sent to the server. Only derived numbers (commit count, file count, branch name, file path of the anchor) and commit SHAs/messages that the user has explicitly linked. This is stated plainly in the README and in `yuzie init`.
- Telemetry is opt-in (`yuzie config set telemetry true`), anonymous, and never includes card titles, branch names, or paths. Off by default.
- `yuzie export` produces a complete dump; account deletion removes all rows within 30 days.
- Self-hosting is a first-class path, documented with a one-file `docker-compose.yml`.

15. Observability

Client. Structured logs to `~/.yuzie/logs/yuzie.log` (rotated, 5 MB). `--verbose` mirrors to `stderr`. Crash reports write a redacted diagnostic bundle (`yuzie doctor --bundle`) containing versions, config with secrets stripped, last 200 log lines, and Git environment facts.

`yuzie doctor` self-check, run by `init` and available anytime:

```
$ yuzie doctor
✓ node v22.4.0 (supported)
✓ git 2.45.1
✓ terminal supports truecolor + unicode
✓ authenticated as @rahul
✓ server reachable (48 ms)
△ hooks: post-commit not installed → run `yuzie hooks install`
✓ cache healthy (312 cards, last sync 12s ago)
```

Server. Pino JSON logs, OpenTelemetry traces on HTTP + WS handlers, Prometheus metrics at `/metrics`: `yuzie_events_total`, `yuzie_ws_connections`, `yuzie_event_lag_seconds`, `yuzie_http_duration_seconds`, `yuzie_conflicts_total`.

16. Testing Strategy

LAYER	TOOLING	WHAT IT COVERS	GATE
Unit	vitest	reducer, rank algebra, slugify, commit parsing, date parsing, formatters	≥ 90% on <code>@yuzie/core</code>
Contract	vitest + zod	Every request/response validates against the shared schema; server and SDK share fixtures	100% of endpoints
DB	vitest + testcontainers (pg)	Migrations up/down, constraint behaviour, seq monotonicity	all migrations
TUI	ink-testing-library	Snapshot of board and card views at 80×24, 120×40; keybinding dispatch	all views
CLI	execa against a local server	Every command's human and <code>--json</code> output, exit codes	all commands
Realtime	vitest + ws harness	Reconnect, replay, gap → snapshot, duplicate suppression, presence expiry	scripted scenarios
Git	temp repos created in fixtures	branch creation, dirty-tree refusal, hook behaviour, commit attribution	all git commands
E2E	two SDK clients + one CLI + server	Journeys A–F from §6, verbatim	all journeys, in CI
Load	k6 / custom	25 clients, 2,000 cards, p95 propagation	pre-release

Golden rule for the coding agent: **a session is not done until its acceptance criteria have tests that fail before the change and pass after.**

17. Packaging & Distribution

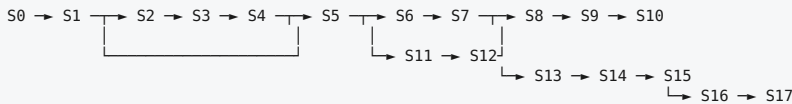
- Published packages: `@yuzie/cli` (bin `yuzie`), `@yuzie/sdk`, `@yuzie/core`, `@yuzie/git`, `@yuzie/mcp`. `yuzie` on npm is an alias package that depends on `@yuzie/cli` so `npm install yuzie` works. (Name availability to be confirmed at Session 0; fallback names: `yuzie-cli`, `yuziehq`, `getyuzie`.)
- Single-file bundles via `tsup`, `engines.node >= 20`, no native deps in the default install path (`better-sqlite3` is an optional dependency with a WASM/JSON fallback so `npm install` never fails on a compile error).
- Versioning: changesets, semver. The `apiVersion` in JSON output and the WS protocol version are versioned independently of the package.
- Release channels: `latest` and `next`. `yuzie upgrade` self-checks and prints the install command.
- Server ships as `ghcr.io/<org>/yuzie-server` plus a `docker-compose.yml` with Postgres and optional Redis.

18. Build Plan — 18 Claude Code Sessions

18.1 How to run these sessions

- One session ≈ one focused Claude Code run (roughly 1–3 hours of agent + review time). Do not merge sessions.
- Each session starts from a green `main: pnpm install && pnpm turbo build test lint typecheck` must pass before you begin.
- Work on a branch `session/NN-<slug>`, open a PR, review the diff yourself, merge.
- If a session's acceptance criteria cannot be met, **stop and split the session** rather than letting it sprawl.
- Keep `SPEC.md` (this document) in the repo root. Every session prompt should begin by telling the agent to read the relevant sections.

Dependency graph



Session 0 — Repository, toolchain, and CI

Goal. A monorepo that builds, tests, lints, and type-checks with zero source code in it yet.

Deliverables - pnpm workspace with `packages/{core,sdk,git,store,cli,server,mcp}` and `e2e/`, each with a minimal `package.json`, `tsconfig.json`, and a passing placeholder test. - Root `tsconfig.base.json` with `strict: true`, `noUncheckedIndexedAccess`, `ESM ("type": "module")`, `moduleResolution: bundler`. - `turbo.json` pipeline: `build → test → lint → typecheck`. - `tsup` config per package; `@yuzie/cli` emits a shebanged single-file bin. - Biome config, `.editorconfig`, `.nvmrc` (Node 22). - Changesets initialised. - GitHub Actions: matrix Node 20/22/24 × ubuntu/macos, running `install → build → test → lint → typecheck`. - `SPEC.md` (this document) committed at root; `README.md` skeleton with the pitch from §2. - `docker-compose.yml` for Postgres + Redis (dev only).

Acceptance - `pnpm i && pnpm turbo build test lint typecheck` green on a clean clone. - `node packages/cli/dist/index.js --version` prints a version. - CI green on a PR.

Claude Code prompt

Read `SPEC.md` sections 10.2, 10.3, and 18 Session 0.

Set up a pnpm + turborepo TypeScript monorepo exactly matching the layout in §10.2. Every package is ESM, strict TypeScript, built with `tsup`, tested with `vitest`, linted with `biome`. `packages/cli` must build to a single shebanged executable at `packages/cli/dist/index.js` that prints its version.

Add GitHub Actions CI (matrix: node 20/22/24 on ubuntu + macos) running `install, build, test, lint, typecheck`. Initialise changesets. Add a `docker-compose.yml` with postgres 16 and redis 7 for local dev.

Do not implement any product logic yet. Each package gets one placeholder test so the pipeline is genuinely exercised. Finish by running the full pipeline and showing me the output.

Session 1 — @yuzie/core: types, schemas, events, reducer

Goal. The single source of truth for the domain, with no I/O.

Deliverables - `types.ts` — `User`, `Board`, `Column`, `Card`, `Comment`, `CheckListItem`, `GitSummary`, `Anchor`, `Presence`, `Role`, `ColumnSemantics`. - `schema.ts` — zod schemas for every entity and every API request/response body in §12.1, exported both as schemas and inferred types. - `events.ts` — a discriminated union of every event in §12.3 with payload schemas; `EventEnvelope { seq, type, actor, cardNo?, payload, ts }`. - `reducer.ts` — `applyEvent(state: BoardState, event: EventEnvelope): BoardState`, pure, immutable, exhaustive over the union, and total (unknown event types are ignored, not thrown). - `rank.ts` — fractional index: `rankBetween(a?: string, b?: string): string`, `rankFirst`, `rankLast`, plus a rebalance helper. - `slug.ts` — `title → branch slug` per §9.2; `branchFor(card, template)`. - `errors.ts` — typed error classes and the error-code union from §12.1.

Acceptance - `pnpm --filter @yuzie/core test` ≥ 90% line coverage. - Property test: 1,000 random `rankBetween` insertions preserve ordering and never produce duplicates. - Property test: applying a shuffled-but-sequential event log yields the same state as applying it in order after sorting by `seq`. - `branchFor` produces `task/18-fix-github-oauth` for the canonical example, and handles emoji, non-Latin scripts, and 200-char titles.

Claude Code prompt

```
Read SPEC.md §11 (data model), §12.3 (event catalogue), §9.2 (branch naming).

Implement packages/core with: types.ts, schema.ts (zod, one schema per
entity and per API body in §12.1), events.ts (discriminated union +
EventEnvelope), reducer.ts (pure applyEvent, exhaustive, total), rank.ts
(fractional indexing), slug.ts (branchFor/slugify), errors.ts.

No I/O, no node built-ins beyond crypto for ids. All exports typed, no `any`.
Write vitest tests including property tests for rank ordering and reducer
associativity as described in Session 1 acceptance criteria. Target 90%
coverage and show me the coverage report at the end.
```

Session 2 — @yuzie/store : local SQLite cache and outbox

Goal. Fast local persistence so the CLI can render before the network answers.

Deliverables - better-sqlite3 (optional dep) with a pure-JSON fallback driver behind the same interface. - Schema + migrations for the tables in §11.3. - CardStore, EventStore, SyncState, Outbox repositories with typed methods. - applyEventToCache(event) reusing @yuzie/core's reducer semantics. - Outbox: enqueue(op), drain(fn), retry/backoff bookkeeping, idempotency keys. - Cache location resolution: .yuzie/cache/yuzie.db in-repo, ~/.yuzie/cache/<slug>.db otherwise.

Acceptance - Reading 2,000 cards from cache takes < 20 ms. - Killing the process mid-write leaves a readable database (WAL mode, transactions). - Outbox survives restart; drain is idempotent given the same keys. - Fallback driver passes the identical test suite.

Claude Code prompt

```
Read SPEC.md §11.3 and §18 Session 2.

Implement packages/store: a local cache over better-sqlite3 (optional
dependency) with a JSON-file fallback driver implementing the same
interface, so `npx yuzie` never fails when native compilation is
unavailable. Include migrations, repositories (cards, columns, comments,
checklist, git, events, sync_state, outbox), WAL mode, and transactional
writes.

Write the same test suite against both drivers via a shared conformance
test. Include a benchmark test asserting a 2000-card read stays under 20ms
on the sqlite driver.
```

Session 3 — @yuzie/server : REST API and persistence

Goal. The authoritative service, without realtime yet.

Deliverables - Fastify app, zod-validated routes from §12.1 (auth, boards, columns, cards, comments, checklist, git, members, tokens). - Drizzle schema + migrations for §11.2, targeting Postgres, with a SQLite variant for self-host. - Device-code auth flow + bearer tokens (sha256-hashed), membership-scoped authorisation middleware, role checks per §14.2. - Append-to events inside the same transaction as every mutation; per-board seq allocation that is monotonic under concurrency (advisory lock or SELECT ... FOR UPDATE on the board row). - Idempotency-key middleware with 24 h store. - Optimistic concurrency via card version / If-Match, returning 409 with current state. - GET /events?since= replay endpoint. - Rate limiting; uniform error envelope; /healthz, /metrics.

Acceptance - Contract tests: every endpoint validates against @yuzie/core schemas in both directions. - Concurrency test: 50 parallel card creations produce 50 distinct card numbers and 50 gapless seq values. - Conflict test: two PATCHes with the same version → one 200, one 409 carrying current state. - Idempotency test: the same key replayed 10× creates one card. - Permission test matrix mirroring §14.2, every cell asserted.

Claude Code prompt

```
Read SPEC.md §11.2, §12.1, §14.2, and §18 Session 3.

Implement packages/server: Fastify + drizzle-orm + postgres. All routes in
§12.1 except the WebSocket stream. Use the zod schemas from @yuzie/core for
validation – do not redefine them. Every mutation appends to the events
table in the same transaction and allocates a strictly monotonic per-board
seq. Implement device-code auth, hashed bearer tokens, role-based
authorisation exactly as in the §14.2 matrix, Idempotency-Key middleware,
If-Match optimistic concurrency returning 409 with current state, rate
limiting, and the uniform error envelope from §12.1.

Test with testcontainers postgres. Include the concurrency, conflict,
idempotency, and permission-matrix tests listed in Session 3 acceptance.
```

Session 4 — Realtime gateway: WebSocket, presence, replay

Goal. Sub-250 ms fan-out with correct resume semantics.

Deliverables - `WS /boards/:slug/stream` implementing §12.2 frames. - Subscription registry keyed by board; broadcast on event commit (in-process `EventEmitter`; Redis pub/sub adapter behind an interface for multi-node). - `hello/welcome` with `since` handling: replay when the gap ≤ 500 , `snapshot` otherwise. - Presence store with 60 s TTL, heartbeat handling, `presence` broadcast coalesced at ≤ 5 Hz. - Backpressure: per-connection outbound queue with a cap; slow clients get a `snapshot` and a reset rather than unbounded buffering. - Auth on upgrade; connection limits per user.

Acceptance - Two clients: a mutation on A reaches B in < 250 ms locally (test asserts < 100 ms). - Kill B's socket, mutate 10 $\times$, reconnect with `since` $\rightarrow$ B receives exactly the 10 missed events, in order, once. - Mutate 600 $\times$ while disconnected $\rightarrow$ B receives a `snapshot`, not a replay. - Presence expires within 60–70 s of a silent disconnect. - 25 concurrent clients, 100 events/s, no dropped or duplicated events.

Claude Code prompt

Read SPEC.md §12.2, §12.3, and §18 Session 4.

Add the WebSocket gateway to `packages/server` implementing the frame protocol in §12.2 exactly: `hello/welcome/event/snapshot/presence/ping/pong`, monotonic seq, replay-on-resume with a 500-event threshold before falling back to `snapshot`, `presence` with 60s TTL and coalesced broadcast, per-connection outbound backpressure, auth on upgrade.

Abstract the fan-out behind a `PubSub` interface with an in-process implementation and a Redis implementation. Write the five acceptance tests listed for Session 4, including the 25-client / 100-events-per-second soak test.

Session 5 — @yuzie/sdk

Goal. A clean, typed client that the CLI and every future client consume.

Deliverables - `Yuzie.connect(slug, opts)` returning a client with `cards`, `boards`, `members`, `comments` resources exactly as §13.1. - HTTP layer: typed fetch wrapper, auth injection, retries with backoff on 5xx/429, typed error mapping. - Realtime layer: WS connection management, reconnect with jitter, `on(type, handler)`, `onPresence`, `close()`. - Optimistic local state driven by `@yuzie/core`'s reducer and `@yuzie/store`'s cache; `board.state` is always readable synchronously. - Offline mode: when `offline: "queue"`, writes go to the outbox and resolve locally; `sync()` drains them with idempotency keys. - Credential resolution per §13.3 (`env` $\rightarrow$ `keychain` $\rightarrow$ `file`).

Acceptance - The SDK compiles and runs in Node and in a browser-like environment (jsdom smoke test) with no Node-only imports in the core path. - Offline test: disconnect, perform 5 writes, reconnect, `sync()` $\rightarrow$ server state matches, no duplicates. - Conflict test: a 409 from the server rolls back optimistic state and emits a `conflict` event. - Public API has zero `any`; `tsd` type tests pass.

Claude Code prompt

Read SPEC.md §13.1, §13.3, §12, and §18 Session 5.

Implement `packages/sdk` exactly to the API shown in §13.1. Layers: `http.ts` (typed fetch, auth, retry/backoff, error mapping to typed classes), `realtime.ts` (WS lifecycle, reconnect with jittered backoff, event subscription), outbox integration with `@yuzie/store` for offline queueing, and resource modules for `cards/boards/comments/members`.

Local state must be maintained via `@yuzie/core`'s reducer so `board.state` is synchronously readable and optimistic updates roll back on 409. No Node-only imports in the core path – it must run in a browser. Add `tsd` type tests and the offline + conflict tests from Session 5 acceptance.

Session 6 — CLI skeleton, config, auth, yuzie init

Goal. Journey A (§6.1) works end to end.

Deliverables - `commander` program, global flags from §7.1, `--help` output that reads well. - Config load/merge/validate (`.yuzie/config.json`, `env`, flags) with zod; `yuzie config get|set`. - `yuzie login` (device flow, keychain storage), `yuzie logout`, `yuzie whoami`. - `yuzie init:repo` detection, remote parsing, auth prompt, board create-or-link, config write, `.gitignore` update, hooks install invocation, `doctor` self-check. - `yuzie doctor` per §15. - Output plumbing: colour handling (`NO_COLOR`, non-TTY), spinner abstraction that no-ops under `--json`, the exit-code table from §7.4 as a single `exit(code)` helper.

Acceptance - Journey A reproduced verbatim in an e2e test against a local server, including the printed receipt. - Running any command unauthenticated exits 3 with a one-line, actionable message. - `--json` on every implemented command emits valid JSON and nothing else on stdout. - Init is idempotent: running twice does not duplicate gitignore lines or hooks.

Claude Code prompt

```
Read SPEC.md §6.1, §7.1, §7.4, §13.2, §13.3, §15, and §18 Session 6.

Implement packages/cli: commander program with the global flags in §7.1,
zod-validated layered config, device-code login storing tokens via
@napi-rs/keyring with a 0600 file fallback, whoami, logout, doctor, and
'yuzie init' reproducing Journey A §6.1 exactly – including the printed
output, .gitignore handling, and idempotency on re-run.

All output goes through a render layer that supports human and --json
modes; stdout in --json mode must contain only JSON. Use the exit codes in
§7.4 via one helper. The CLI must talk to the server only through
@yuzie/sdk. E2E-test Journey A against a locally started server.
```

Session 7 — Core CLI commands

Goal. The whole non-interactive command surface from §7.2 (except Git-specific commands).

Deliverables - `add`, `list`, `card`, `move`, `assign`, `done`, `comment`, `edit`, `rm`, `watch`, `check`, `label`, `due`, `priority`. - `boards`, `columns`, `members`, `invite`, `who`, `activity`, `feed`. - Card ID resolution per §7.5 (number, `#n`, fuzzy title, disambiguation prompt). - Human formatters: the table in §7.3, width-aware truncation, relative times, symbol set; and JSON formatters with the `apiVersion / kind / data / meta` envelope. - Natural-language date parsing for `--due` (`friday`, `tomorrow`, `+3d`, ISO). - `yuzie edit` round-trip through `$EDITOR` as YAML front-matter + markdown body, producing a minimal PATCH. - `yuzie feed` streaming live events to stdout, line-per-event, `--json` supported.

Acceptance - Every command has: a human-output test, a `--json` schema-validated test, and an exit-code test. - `yuzie list` output matches the §7.3 sample byte-for-byte given the fixture data. - `yuzie list --json | jq '.data | length'` works in a shell test. - Fuzzy resolution: ambiguous prefix prompts interactively, exits 4 under `--json / --yes`.

Claude Code prompt

```
Read SPEC.md §7.2, §7.3, §7.5, and §18 Session 7.

Implement every non-Git command in the §7.2 tables. Build two formatter
families in src/render: human (width-aware, colourised, relative times, the
exact table layout in §7.3) and json (apiVersion/kind/data/meta envelope,
schema-validated against @yuzie/core).

Implement card ID resolution per §7.5 including fuzzy title matching with
interactive disambiguation, which must exit 4 instead of prompting when
--json or --yes is set. Implement 'yuzie edit' as an $EDITOR round-trip
producing a minimal PATCH, and 'yuzie feed' as a blocking live event
stream. Test every command in all three dimensions listed in Session 7
acceptance.
```

Session 8 — TUI foundation: board view

Goal. `yuzie` with no arguments renders the board and navigates.

Deliverables - Ink app bootstrap, alternate screen buffer, clean teardown on `q / Ctrl-C / SIGTERM`. - Layout engine: column widths, horizontal overflow with `< >`, vertical scrolling, the frame in §8.2. - Card cell renderer with id, title truncation, assignees, presence dot, git badge, staleness, priority/label chips. - Navigation state machine (`columns/cards`, `h j k l`, arrows, `g g / G`, `1 – 9`). - Responsive fallback to single-column list under 100 cols (§8.6). - Theme module: truecolor / 256 / no-colour tiers; unicode / ASCII tiers. - Render from cache first, then patch when the SDK's state updates.

Acceptance - ink-testing-library snapshots at 80×24, 100×30, 160×50 for: empty board, 3-column board, 12-column board, 200-card column. - First paint from warm cache under 400 ms (timed test). - Every keybinding in the board column of §8.4 dispatches the right action (unit-tested through the state machine, not the renderer). - No crash and no visual corruption on terminal resize (simulated).

Claude Code prompt

Read SPEC.md §8.1, §8.2, §8.4, §8.6, and §18 Session 8.

Implement the Ink TUI board view in `packages/cli/src/tui`. Render the layout in §8.2 including the header status line and the footer hint bar. Implement the navigation state machine as a pure reducer separate from the React components so it can be unit tested without rendering.

Support horizontal column overflow, vertical scroll, resize, the sub-100-column single-column fallback, and colour/unicode capability tiers. Render from the local cache immediately, then patch from SDK state. Add `ink-testing-library` snapshot tests at the three terminal sizes and the four board fixtures listed in Session 8 acceptance.

Session 9 — TUI card detail, modals, and editing

Goal. The card view in §8.3 and every interactive mutation.

Deliverables - Card detail view with all panels from §8.3. - Modals/overlays: column picker (`m`), member picker (`a`), comment input (`C` , multiline, `Ctrl-D` to send), new-card inline prompt (`n`), confirm dialog (`D`), help overlay (`?`), search/filter (`/` , `f`). - Checklist interaction (`x` toggles, `+` adds). - Suspend-and-resume for `$EDITOR` (`e`): drop out of the alt screen, run the editor, restore cleanly. - Optimistic UI: every action paints immediately, marks pending with a subtle indicator, reconciles or rolls back.

Acceptance - Snapshot tests for the card view with: no description, long description, 20 checklist items, 50 activity entries, no git link. - Editor suspend/resume test using a fake `$EDITOR` script; terminal state fully restored afterwards. - Every card-view keybinding in §8.4 tested through the state machine. - Rollback test: a mutation that 409s visibly reverts and shows the conflict marker.

Claude Code prompt

Read SPEC.md §8.3, §8.4, and §18 Session 9.

Implement the TUI card detail view and all interactive overlays: column picker, member picker, comment composer, new-card prompt, confirm dialog, help overlay, search/filter. Implement checklist toggling and `$EDITOR` suspend/resume that leaves the terminal in a clean state even if the editor exits non-zero.

All mutations are optimistic with a pending indicator and roll back on conflict. Extend the pure navigation reducer rather than putting logic in components. Add the snapshot and behaviour tests listed in Session 9 acceptance.

Session 10 — Realtime in the TUI: live updates and presence

Goal. Journey D (§6.4) — the moment the product sells itself.

Deliverables - SDK event subscription wired into the TUI with a coalescing frame budget (batch events, re-render at most 20 fps). - Presence rendering per §8.5: dots, “is viewing”, “is working”, agent styling. - Presence emission: `viewing` on card focus, `working` derived from claim + checked-out branch, cleared on exit. - Toast system in the footer: 3-second transient event notices, queue with a cap of 3. - Connection status in the header: `synced` / `△ reconnecting...` / `△ offline` · `N queued`, never blocking input. - Card-level animation affordance: moved cards flash once; conflicted cards show `⌕ updated by @x`.

Acceptance - Two-client integration test: client A moves a card, client B's rendered output contains the moved card and the toast within 500 ms. - Presence test: A opens card 18, B sees “● @a is viewing” in the card footer. - Disconnect test: server killed → header shows offline, board stays navigable, writes queue, and on restart everything reconciles with no duplicates. - 100 events in 1 second causes at most 20 renders.

Claude Code prompt

Read SPEC.md §6.4, §8.5, §12.2, and §18 Session 10.

Wire `@yuzie/sdk` realtime into the TUI. Batch incoming events into at most 20 renders per second. Implement presence emission (`viewing/working`) and presence rendering per §8.5, the footer toast queue, and the header connection status with offline/queued counts.

Write a two-client integration test that starts a real local server, runs one client via the SDK and one via the rendered TUI, and asserts the propagation and toast behaviour of Journey D §6.4 within 500ms. Include the disconnect/reconcile and render-budget tests from Session 10 acceptance.

Session 11 — `@yuzie/git`: repo introspection and the claim flow

Goal. The differentiator. Journey B (§6.2).

Deliverables - `repo.ts`: detection (walk up for `.git`), remote parsing (SSH + HTTPS + GitHub/GitLab), current branch, dirty state, detached HEAD, base-branch detection. - `branch.ts`: `branchFor` (from core), create/checkout/track, existence checks local and remote. - `commits.ts`: `git log base..branch` parsing, diff stats, commit→card resolution order per §9.6. - `hooks.ts`: idempotent install/uninstall, append-safe if hooks already exist, generated shims that call `yuzie __hook <name>`, hard-wrapped so they can never fail a Git operation. - `editor.ts`: editor resolution and line-jump command table per §9.7. - CLI commands: `claim`, `start`, `finish`, `branch`, `commits`, `hooks`, and the hidden `__hook` handler. - Preflight logic for `claim` (§9.3) and `finish` (§9.4) with exact prompts and exit code 8.

Acceptance - Fixture repos created in temp dirs by a helper; tests cover clean tree, dirty tree, detached HEAD, existing local branch, existing remote branch, no remote, non-repo directory. - `claim` on a dirty tree exits 8 under `--yes`, prompts interactively otherwise. - `post-commit` hook attaches a commit to the right card via each of the four resolution rules in §9.6, and exits 0 even when the server is unreachable. - Journey B reproduced as an e2e test, including the printed receipt lines. - Hook install run twice produces identical hook files.

Claude Code prompt

```
Read SPEC.md §9 entirely, §6.2, and §18 Session 11.

Implement packages/git (repo detection, remote parsing, branch operations,
commit and diff parsing, commit-card resolution per §9.6, hook management,
editor resolution) and the CLI commands claim, start, finish, branch,
commits, hooks, plus the hidden `yuzie __hook <name>` handler.

Implement the claim preflight (§9.3) and finish preflight (§9.4) exactly,
including prompts, warnings, --force/--skip-checks, and exit code 8. Git
hooks must be idempotent, append-safe, time-limited, and must always exit 0.

Build a temp-repo fixture helper and cover every repo state listed in
Session 11 acceptance. Finish with an e2e test reproducing Journey B §6.2.
```

Session 12 — Code anchors, open-in-editor, open-in-browser

Goal. Journey F (§6.5) — task to exact line in one keystroke.

Deliverables - `yuzie anchor <id> <file:line[-endLine]>` with path normalisation relative to repo root and existence validation. - Anchor staleness detection: compare stored `commitSha` against current file history; render `⚠ anchor may be stale`. - `yuzie open <id>` variants: editor (default), `--github` (branch or PR compare URL built from the remote), `--pr`, `--browser` (attached URL). - PR discovery: `gh` CLI if available, else GitHub REST with the user's token, else null — all failures degrade silently to branch compare. - TUI `o` and `g` bindings wired to the same code path.

Acceptance - Editor resolution tested for `code`, `cursor`, `nvim`, `vim`, `subl`, and an unknown `$EDITOR` (must fail with a helpful message, exit 1). - URL construction tested for GitHub SSH remotes, HTTPS remotes, GitLab, and self-hosted remotes with ports. - Stale-anchor detection test using a fixture repo where the file changed after the anchor commit. - `yuzie open` never opens anything in a non-TTY/CI environment; it prints the target instead.

Claude Code prompt

```
Read SPEC.md §9.7, §6.5, and §18 Session 12.

Implement code anchors and the open commands: `yuzie anchor`, `yuzie open`
with editor/--github/--pr/--browser variants, PR discovery via gh CLI with
a GitHub REST fallback and silent degradation, remote-URL to web-URL
construction for GitHub/GitLab/self-hosted (SSH and HTTPS forms), and
anchor staleness detection against the stored commit sha.

Wire the TUI `o` and `g` keys to the same code paths. In non-TTY or CI
environments, print the target instead of launching anything. Cover every
case in Session 12 acceptance.
```

Session 13 — Offline mode and the sync engine

Goal. The tool is never blocked by the network.

Deliverables - Connectivity detection with a fast fail (250 ms budget) and an explicit `--offline` flag. - Outbox drain on reconnect: ordered, idempotent, with per-op retry limits and a poison-op quarantine surfaced by `yuzie doctor`. - `yuzie sync`: push outbox, pull events since `last_seq` (or snapshot), re-scan Git summaries for cards with linked branches, print a reconciliation report. - Conflict surfacing: `yuzie sync` lists cards whose optimistic state was replaced. - Cache invalidation and repair (`yuzie sync --rebuild`).

Acceptance - Network-blackhole e2e: 10 mixed writes offline, reconnect, `sync` → server state exactly matches expectation, zero duplicates, event log has 10 new entries. - Poison op (a write to a deleted card) is quarantined after 3 attempts, reported clearly, and does not block the rest of the queue. - Concurrent-offline test: two clients both edit card 18 offline; on sync, one wins, the other is told exactly what happened. - `yuzie list` works with the server down, and clearly labels the data as cached with an age.

Claude Code prompt

```
Read SPEC.md §11.3, §11.4, §12.1 (idempotency), and §18 Session 13.

Implement the sync engine: connectivity detection with a 250ms budget,
ordered idempotent outbox drain on reconnect, poison-op quarantine after 3
attempts, `yuzie sync` with a printed reconciliation report, conflict
surfacing, and `yuzie sync --rebuild` for cache repair.

Every offline write must resolve locally and be labelled as queued in all
output modes. Every read must work from cache with a visible age label when
offline. Write the four acceptance tests for Session 13, including a
network-blackhole harness.
```

Session 14 — Watching, activity, notifications, search

Goal. Awareness without noise.

Deliverables - `watch / unwatch`, auto-watch on comment or assignment (configurable). - `yuzie activity` with `--since`, `--card`, `--author`, `--json`; rendered as a projection over the event log. - TUI activity panel and a board-level activity drawer. - Terminal notifications for watched cards while the TUI is open (footer toast + optional OS notification via `node-notifier`, off by default). - Search: `/` in the TUI and `yuzie list --search`, matching title, description, comments, labels, assignee; server-side `ILIKE` /FTS with a local fallback when offline. - Filters: `--mine`, `--watching`, `--stale <dur>`, `--label`, `--assignee`, `--column`; saved as a session filter in the TUI with `f`.

Acceptance - Search returns correct results for 2,000-card fixtures in under 100 ms server-side and under 200 ms from cache. - Watch test: B watches #18, A comments, B's TUI shows a toast and `yuzie activity --card 18` includes it. - Stale filter correctly computes "claimed N days ago with no commits" from git summaries plus event history. - Activity output is stable and paginated; `--json` validates.

Claude Code prompt

```
Read SPEC.md §7.2 (team & awareness commands), §12.3, and §18 Session 14.

Implement watching (with configurable auto-watch), the activity feed as a
projection over the event log (CLI + TUI panel), footer toasts for watched
cards with optional OS notifications defaulting to off, and search plus the
filter set in §7.2 across both the CLI and the TUI's `/` and `f` keys.

Search must work server-side and fall back to the local cache when offline.
Implement the `--stale` computation from git summaries plus claim events.
Meet the latency numbers in Session 14 acceptance and test them.
```

Session 15 — Agent integration: tokens and MCP server

Goal. Journey E (§6.5) — an agent as a first-class board participant.

Deliverables - `yuzie token create/list/revoke` with role and optional board scope; plaintext shown exactly once. - Agent user kind: created on first agent-token use, rendered with `(agent)` everywhere, distinct presence colour. - `packages/mcp`: an MCP stdio server exposing the tools in §13.4, built on `@yuzie/sdk`. - Guardrails: destructive tools disabled unless `--allow-destructive`; git-mutating tools disabled unless `--allow-git`; every agent action attributed and logged. - Docs: a copy-paste MCP config block for Claude Code and a short "agent etiquette" section (narrate progress via comments, tick checklist items, never silently delete).

Acceptance - MCP protocol conformance test: initialise, list tools, call each tool, error paths. - An agent token cannot exceed its role; scoped tokens cannot touch other boards (asserted per permission cell). - Journey E reproduced as an e2e test with a scripted agent, including presence showing `● @claude ● working (agent)` on a human client. - Destructive tool call without the flag returns a clear refusal, not a crash.

Claude Code prompt

Read SPEC.md §13.4, §14.2, §6.5, and §18 Session 15.

Implement scoped API tokens in the CLI and server (create/list/revoke, plaintext shown once, sha256 storage, optional board scope), the agent user kind with distinct rendering and presence, and packages/mcp: an MCP stdio server built on @yuzie/sdk exposing exactly the tools in §13.4 with the --allow-destructive and --allow-git guardrails.

Write MCP protocol conformance tests, per-cell token permission tests, and an e2e test reproducing Journey E where a scripted agent claims a card and a human client observes the agent's presence and comments.

Session 16 — Hardening: performance, resilience, and diagnostics

Goal. Make it feel fast and trustworthy under real conditions.

Deliverables - Startup profiling and lazy-loading: defer Ink, SQLite, and Git modules until needed so one-shot commands hit the < 150 ms budget. - Bundle-size budget check in CI. - Load test suite: 25 clients × 2,000 cards × 100 events/s, asserting p95 propagation and server memory. - Memory profiling of the TUI on a 500-card board. - Error handling audit: every `catch` produces a typed error with an actionable message; no raw stack traces to users without `--verbose`. - `yuzie doctor --bundle` diagnostic bundle with redaction. - Log rotation, crash handler, graceful SIGINT/SIGTERM everywhere (including hooks and the TUI alt screen).

Acceptance - `hyperfine 'yuzie list --json'` p50 under 150 ms with a warm cache, enforced in CI. - Load test meets §10.4 targets and is runnable with one command. - Fuzzing the CLI with malformed args and hostile inputs (unicode, 10 KB titles, control characters) produces no crashes and no corrupted output. - Every error path has a test asserting exit code, stderr message, and absence of a stack trace.

Claude Code prompt

Read SPEC.md §10.4, §15, and §18 Session 16.

Harden the product: lazy-load heavy modules so one-shot CLI commands start in under 150ms (enforce with a hyperfine check in CI), add a bundle-size budget, build a runnable load-test suite meeting the §10.4 targets, audit every error path to produce typed actionable messages with correct exit codes and no user-facing stack traces, implement `'yuzie doctor --bundle'` with secret redaction, and ensure graceful signal handling everywhere including the TUI alt-screen and git hooks.

Fuzz the CLI with hostile inputs (10KB titles, control characters, invalid unicode, path traversal in anchors) and fix everything it breaks.

Session 17 — Docs, packaging, and release

Goal. A stranger can `npx yuzie` and be productive in a minute.

Deliverables - README: the §2 pitch, an animated demo (ascinema/VHS), 60-second quickstart, full command reference generated from commander metadata. - `docs/`: self-hosting guide with `docker-compose.yml`, SDK reference, MCP/agent guide, keybinding cheatsheet, FAQ, privacy statement (§14.3). - `yuzie completion` for bash/zsh/fish, with dynamic completion of card IDs and columns from cache. - Publish pipeline: `changesets` → `npm publish` for all packages + the `yuzie` alias package; provenance/attestation enabled; server image published to GHCR. - `yuzie upgrade` and a version-check nudge (max once per day, silenceable). - Release checklist and a `CHANGELOG.md` generated by `changesets`.

Acceptance - A clean machine (CI container with only Node) runs `npx yuzie@latest init` in a fresh Git repo and completes Journey A. - Self-host guide followed verbatim in CI brings up a working server and a passing e2e run against it. - All published package `exports` maps resolve under Node ESM, CJS interop, and bundlers (`arethetypeswrong clean`). - `yuzie --help` and `yuzie <cmd> --help` match the documented command reference (test compares generated output to docs).

Claude Code prompt

Read SPEC.md §17, §14.3, and §18 Session 17.

Write the README (pitch from §2, VHS-recorded demo, quickstart, generated command reference) and docs/ (self-hosting with docker-compose, SDK reference, agent/MCP guide, keybinding cheatsheet, privacy statement). Implement shell completions with dynamic card/column completion from cache, `'yuzie upgrade'`, and a once-a-day silenceable version nudge.

Set up the `changesets` release pipeline publishing all packages plus a `'yuzie'` alias package with `npm provenance`, and publish the server image to GHCR. Add CI jobs that (a) run `'npx yuzie@latest init'` on a clean container and complete Journey A, (b) follow the self-hosting guide verbatim and run e2e against it, and (c) verify `exports` maps with `arethetypeswrong`.

19. Milestones

MILESTONE	SESSIONS	DEMO-ABLE OUTCOME
M0 — Foundation	0–2	Monorepo builds; domain model and local cache are tested and solid.
M1 — Server alive	3–4	Two <code>curl / wscat</code> clients see each other's changes in real time.
M2 — CLI usable	5–7	A team could actually use Yuzie through commands alone. Internal dogfooding starts here.
M3 — TUI usable	8–10	Journey D lands: the live multiplayer board. This is the demo that sells it.
M4 — Git-aware	11–12	Journey B and F land. This is the differentiator; start posting build-in-public content here.
M5 — Resilient	13–14	Offline, sync, awareness. Safe for teams on bad connections.
M6 — Agents	15	Journey E: an agent working a card alongside humans.
M7 — Launch	16–17	Public <code>npx yuzie</code> .

Dogfooding rule. From M2 onward, the board's own development is tracked on Yuzie itself. Every subsequent session is a card. This is both the best test suite and the best marketing asset.

20. Risks & Mitigations

RISK	IMPACT	LIKELIHOOD	MITIGATION
Terminal rendering inconsistency across emulators (widths, emoji, box drawing)	High — the product <i>is</i> the rendering	High	Capability tiers (unicode/ASCII, truecolor/256/none), snapshot tests at fixed sizes, a <code>yuzie doctor</code> terminal check, and an explicit supported-terminal list
<code>better-sqlite3</code> native build failures break <code>npx</code>	High	Medium	Optional dependency + JSON fallback driver, conformance-tested (Session 2)
Realtime correctness bugs (dupes, gaps) erode trust fast	High	Medium	Monotonic seq + replay + idempotency keys + the Session 4 test suite as a permanent regression gate
Scope creep into a Trello clone	High	High	§5.3 is a contract. Any new feature must displace something or wait for v1.1
Hosting cost and abuse on a free tier	Medium	Medium	Self-host as the documented default; hosted tier rate-limited and invite-gated at launch
Git hooks annoying or breaking users' workflows	High	Medium	Hooks always exit 0, are time-limited, opt-out at init, and fully documented
npm name unavailable	Low	Medium	Fallback names decided at Session 0 before any branding work
Solo/small-team bandwidth	High	High	18 discrete sessions with hard acceptance criteria; ship M3 as a public teaser even if M5+ slips

21. Post-v1 Roadmap

VERSION	THEME	HIGHLIGHTS
v1.1	Ergonomics	GitHub Issues one-way import, board templates, saved filters, <code>yuzie stats</code> (cycle time, WIP, stale report)
v1.2	Team scale	Multiple repos per board, per-column WIP limits enforced, roles beyond three
v1.5	Beyond the terminal	Read-only web view generated from the SDK, VS Code extension, shareable card permalinks
v2.0	Workflow	Bidirectional issue sync, automation rules (<code>when card enters Review</code> , <code>request review</code>), agent orchestration (a queue of cards agents may pull from)

Appendix A — Full Command Reference (quick card)

SETUP	init · login · logout · whoami · link · unlink · doctor · config
BOARDS	boards [create rename archive] · columns [add rm]
CARDS	add · list · card · move · assign · claim · start · finish · done comment · edit · rm · watch · unwatch · check · label · due · priority
GIT	branch · anchor · open · commits · hooks · sync
TEAM	invite · members · who · activity · feed
AUTOMATION	token · mcp · serve · export · import · completion · upgrade
GLOBAL	--json --yuzie --no-color --quiet --verbose --offline --yes --config

Appendix B — Keybinding Cheatsheet

NAVIGATE	← → h l	columns	↑ ↓ j k	cards
	1-9	jump to column	gg / G	first / last card
ACT	n new	c claim	m move	a assign d done
	C comment	e edit	x check	w watch D delete
OPEN	↵ card	o editor	g github	
VIEW	/ search	f filter	r refresh ? help	q quit

Appendix C — Error Codes

CODE	EXIT	MEANING	TYPICAL FIX
unauthenticated	3	No or expired token	yuzie login
forbidden	5	Role lacks permission	Ask an owner
card_not_found	4	Bad id or wrong board	yuzie list
board_not_found	4	Slug wrong / not a member	yuzie boards
column_not_found	4	Unknown column	yuzie columns
version_conflict	6	Someone else changed it	Re-read and retry
validation_failed	2	Bad input	Check --help
wip_limit_exceeded	1	Column full	Move something out
rate_limited	1	Too many requests	Back off
offline_required_network	7	Needs the server	Reconnect or yuzie sync later
git_precondition_failed	8	Dirty tree, detached HEAD	Commit/stash, or --force

Appendix D — Glossary

TERM	MEANING
Board	A kanban board, usually one per repository.
Card	A unit of work, addressed as #18 within its board.
Column	A stage (Todo/Doing/Review/Done), with optional semantics used by claim / finish .
Claim	Assign to self + move to in-progress + create/checkout a branch.
Anchor	A file:line reference stored on a card.
Rank	A fractional index string controlling order within a column.
Seq	Monotonic per-board event sequence number; the basis of sync.
Outbox	Local queue of writes made while offline.
Presence	Ephemeral state of who is online, viewing, or working.
Agent	A non-human board participant authenticated with a scoped token.

Appendix E — Definition of Done (applies to every session)

A session is complete only when **all** of the following hold:

1. All stated deliverables exist and are exported where the spec says they should be.
2. All acceptance criteria have tests that fail before the change and pass after.
3. `pnpm turbo build test lint typecheck` is green across the whole monorepo.
4. No `any`, no `@ts-ignore`, no `eslint-disable` -equivalent in new code without an inline justification comment.
5. Every user-facing string is actionable: it says what happened and what to do next.
6. `--json` output for any touched command validates against the shared schema.
7. A changeset is added describing the user-visible change.
8. The README or docs are updated if the user-facing surface changed.
9. The session's own card on the board is moved to Done (from M2 onward).